

org.apache.hadoop.fs

Class **FileSystem**

java.lang.Object
org.apache.hadoop.conf.Configured
org.apache.hadoop.fs.FileSystem

All Implemented Interfaces:

Closeable, AutoCloseable, Configurable

Direct Known Subclasses:

FilterFileSystem, FTPFileSystem, NativeAzureFileSystem, NativeS3FileSystem, RawLocalFileSystem, S3FileSystem, ViewFileSystem

```
@InterfaceAudience.Public
@InterfaceStability.Stable
public abstract class FileSystem
extends Configured
implements Closeable
```

An abstract base class for a fairly generic filesystem. It may be implemented as a distributed filesystem, or as a "local" one that reflects the locally-connected disk. The local version exists for small Hadoop instances and for testing.

All user code that may potentially use the Hadoop Distributed File System should be written to use a FileSystem object. The Hadoop DFS is a multi-machine system that appears as a single disk. It's useful because of its fault tolerance and potentially very large capacity.

The local implementation is [LocalFileSystem](#) and distributed implementation is [DistributedFileSystem](#).

Field Summary

Fields

Modifier and Type	Field and Description
static String	DEFAULT_FS
static String	FS_DEFAULT_NAME_KEY
static org.apache.commons.logging.Log	LOG
static int	SHUTDOWN_HOOK_PRIORITY
	Priority of the FileSystem shutdown hook.
protected org.apache.hadoop.fs.FileSystem.Statistics	statistics
	The statistics for this file system.

Constructor Summary

Constructors

Modifier	Constructor and Description
protected	FileSystem()

Method Summary

Methods

Modifier and Type	Method and Description
FSDaataOutputStream	append(Path f) Append to an existing file (optional operation).
FSDaataOutputStream	append(Path f, int bufferSize) Append to an existing file (optional operation).
abstract FSDaataOutputStream	append(Path f, int bufferSize, Progressable progress) Append to an existing file (optional operation).
static boolean	areSymlinksEnabled()
boolean	cancelDeleteOnExit(Path f) Cancel the deletion of the path when the FileSystem is closed
protected URI	canonicalizeUri(URI uri) Canonicalize the given URI.
protected void	checkPath(Path path)

	Check that a Path belongs to this FileSystem.
static void	<code>clearStatistics()</code> Reset all statistics for all file systems
void	<code>close()</code> No more filesystem operations are needed.
static void	<code>closeAll()</code> Close all cached filesystems.
static void	<code>closeAllForUGI(org.apache.hadoop.security.UserGroupInformation ugi)</code> Close all cached filesystems for a given UGI.
void	<code>completeLocalOutput(Path fsOutputFile, Path tmpLocalFile)</code> Called when we're all done writing to the target.
void	<code>concat(Path trg, Path[] psrCs)</code> Concat existing files together.
void	<code>copyFromLocalFile(boolean delSrc, boolean overwrite, Path[] srcs, Path dst)</code> The src files are on the local disk.
void	<code>copyFromLocalFile(boolean delSrc, boolean overwrite, Path src, Path dst)</code> The src file is on the local disk.
void	<code>copyFromLocalFile(boolean delSrc, Path src, Path dst)</code> The src file is on the local disk.
void	<code>copyFromLocalFile(Path src, Path dst)</code> The src file is on the local disk.
void	<code>copyToLocalFile(boolean delSrc, Path src, Path dst)</code> The src file is under FS, and the dst is on the local disk.
void	<code>copyToLocalFile(boolean delSrc, Path src, Path dst, boolean useRawLocalFileSystem)</code> The src file is under FS, and the dst is on the local disk.
void	<code>copyToLocalFile(Path src, Path dst)</code> The src file is under FS, and the dst is on the local disk.
static FSDataOutputStream	<code>create(FileSystem fs, Path file, FsPermission permission)</code> create a file with the provided permission The permission of the file is set to be the provided permission as in setPermission, not permission&~umask It is implemented using two RPCs.
FSDataOutputStream	<code>create(Path f)</code> Create an FSDataOutputStream at the indicated Path.
FSDataOutputStream	<code>create(Path f, boolean overwrite)</code> Create an FSDataOutputStream at the indicated Path.
FSDataOutputStream	<code>create(Path f, boolean overwrite, int bufferSize)</code> Create an FSDataOutputStream at the indicated Path.
FSDataOutputStream	<code>create(Path f, boolean overwrite, int bufferSize, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
FSDataOutputStream	<code>create(Path f, boolean overwrite, int bufferSize, short replication, long blockSize)</code> Create an FSDataOutputStream at the indicated Path.
FSDataOutputStream	<code>create(Path f, boolean overwrite, int bufferSize, short replication, long blockSize, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
abstract FSDataOutputStream	<code>create(Path f, FsPermission permission, boolean overwrite, int bufferSize, short replication, long blockSize, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
FSDataOutputStream	<code>create(Path f, FsPermission permission, EnumSet<CreateFlag> flags, int bufferSize, short replication, long blockSize, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
FSDataOutputStream	<code>create(Path f, FsPermission permission, EnumSet<CreateFlag> flags, int bufferSize, short replication, long blockSize, Progressable progress, org.apache.hadoop.fs.Options.ChecksumOpt checksumOpt)</code> Create an FSDataOutputStream at the indicated Path with a custom checksum option
FSDataOutputStream	<code>create(Path f, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
FSDataOutputStream	<code>create(Path f, short replication)</code> Create an FSDataOutputStream at the indicated Path.
FSDataOutputStream	<code>create(Path f, short replication, Progressable progress)</code> Create an FSDataOutputStream at the indicated Path with write-progress reporting.
boolean	<code>createNewFile(Path f)</code>

	Creates the given Path as a brand-new zero-length file.
<code>FSDataOutputStream</code>	<code>createNonRecursive(Path f, boolean overwrite, int bufferSize, short replication, long blockSize, Progressable progress)</code> Deprecated. <i>API only for 0.20-append</i>
<code>FSDataOutputStream</code>	<code>createNonRecursive(Path f, FsPermission permission, boolean overwrite, int bufferSize, short replication, long blockSize, Progressable progress)</code> Deprecated. <i>API only for 0.20-append</i>
<code>FSDataOutputStream</code>	<code>createNonRecursive(Path f, FsPermission permission, EnumSet<CreateFlag> flags, int bufferSize, short replication, long blockSize, Progressable progress)</code> Deprecated. <i>API only for 0.20-append</i>
<code>Path</code>	<code>createSnapshot(Path path)</code> Create a snapshot with a default name.
<code>Path</code>	<code>createSnapshot(Path path, String snapshotName)</code> Create a snapshot
<code>void</code>	<code>createSymlink(Path target, Path link, boolean createParent)</code> See <code>FileContext.createSymlink(Path, Path, boolean)</code>
<code>boolean</code>	<code>delete(Path f)</code> Deprecated. <i>Use <code>delete(Path, boolean)</code> instead.</i>
<code>abstract boolean</code>	<code>delete(Path f, boolean recursive)</code> Delete a file.
<code>boolean</code>	<code>deleteOnExit(Path f)</code> Mark a path to be deleted when FileSystem is closed.
<code>void</code>	<code>deleteSnapshot(Path path, String snapshotName)</code> Delete a snapshot of a directory
<code>static void</code>	<code>enableSymlinks()</code>
<code>boolean</code>	<code>exists(Path f)</code> Check if exists.
<code>protected Path</code>	<code>fixRelativePart(Path p)</code> See <code>FileContext.fixRelativePart(org.apache.hadoop.fs.Path)</code>
<code>static FileSystem</code>	<code>get(Configuration conf)</code> Returns the configured filesystem implementation.
<code>static FileSystem</code>	<code>get(URI uri, Configuration conf)</code> Returns the FileSystem for this URI's scheme and authority.
<code>static FileSystem</code>	<code>get(URI uri, Configuration conf, String user)</code> Get a filesystem instance based on the uri, the passed configuration and the user
<code>AclStatus</code>	<code>getAclStatus(Path path)</code> Gets the ACL of a file or directory.
<code>static List<org.apache.hadoop.fs.FileSystem.Statistics></code>	<code>getAllStatistics()</code> Return the FileSystem classes that have Statistics
<code>long</code>	<code>getBlockSize(Path f)</code> Deprecated. <i>Use <code>getFileStatus()</code> instead</i>
<code>protected URI</code>	<code>getCanonicalUri()</code> Return a canonicalized form of this FileSystem's URI.
<code>ContentSummary</code>	<code>getContentSummary(Path f)</code> Return the <code>ContentSummary</code> of a given <code>Path</code> .
<code>long</code>	<code>getDefaultBlockSize()</code> Deprecated. <i>use <code>getDefaultBlockSize(Path)</code> instead</i>
<code>long</code>	<code>getDefaultBlockSize(Path f)</code> Return the number of bytes that large input files should be optimally be split into to minimize i/o time.
<code>protected int</code>	<code>getDefaultPort()</code> Get the default port for this file system.
<code>short</code>	<code>getDefaultReplication()</code> Deprecated. <i>use <code>getDefaultReplication(Path)</code> instead</i>
<code>short</code>	<code>getDefaultReplication(Path path)</code> Get the default replication for a path.
<code>static URI</code>	<code>getDefaultUri(Configuration conf)</code> Get the default filesystem URI from a configuration.
<code>BlockLocation[]</code>	<code>getFileBlockLocations(FileStatus file, long start, long len)</code> Return an array containing hostnames, offset and size of portions of the given file.
<code>BlockLocation[]</code>	<code>getFileBlockLocations(Path p, long start, long len)</code> Return an array containing hostnames, offset and size of portions of the given file.

<code>FileChecksum</code>	<code>getFileChecksum(Path f)</code> Get the checksum of a file.
<code>FileChecksum</code>	<code>getFileChecksum(Path f, long length)</code> Get the checksum of a file, from the beginning of the file till the specific length.
<code>FileStatus</code>	<code>getFileLinkStatus(Path f)</code> See <code>FileContext.getFileLinkStatus(Path)</code>
<code>abstract FileStatus</code>	<code>getFileStatus(Path f)</code> Return a file status object that represents the path.
<code>static Class<? extends FileSystem></code>	<code>getFileSystemClass(String scheme, Configuration conf)</code>
<code>protected static FileSystem</code>	<code>getFSofPath(Path absOrFqPath, Configuration conf)</code>
<code>Path</code>	<code>getHomeDirectory()</code> Return the current user's home directory in this filesystem.
<code>protected Path</code>	<code>getInitialWorkingDirectory()</code> Note: with the new <code>FileContext</code> class, <code>getWorkingDirectory()</code> will be removed.
<code>long</code>	<code>getLength(Path f)</code> Deprecated. Use <code>getFileStatus()</code> instead
<code>Path</code>	<code>getLinkTarget(Path f)</code> See <code>FileContext.getLinkTarget(Path)</code>
<code>static LocalFileSystem</code>	<code>getLocal(Configuration conf)</code> Get the local file system.
<code>String</code>	<code>getName()</code> Deprecated. call <code>#getUri()</code> instead.
<code>static FileSystem</code>	<code>getNamed(String name, Configuration conf)</code> Deprecated. call <code>#get(URI, Configuration)</code> instead.
<code>short</code>	<code>getReplication(Path src)</code> Deprecated. Use <code>getFileStatus()</code> instead
<code>String</code>	<code>getScheme()</code> Return the protocol scheme for the FileSystem.
<code>FsServerDefaults</code>	<code>getServerDefaults()</code> Deprecated. use <code>getServerDefaults(Path)</code> instead
<code>FsServerDefaults</code>	<code>getServerDefaults(Path p)</code> Return a set of server default configuration values
<code>static Map<String,org.apache.hadoop.fs.FileSystem.Statistics></code>	<code>getStatistics()</code> Deprecated. use <code>getAllStatistics()</code> instead
<code>static org.apache.hadoop.fs.FileSystem.Statistics</code>	<code>getStatistics(String scheme, Class<? extends FileSystem> cls)</code> Get the statistics for a particular file system
<code>FsStatus</code>	<code>getStatus()</code> Returns a status object describing the use and capacity of the file system.
<code>FsStatus</code>	<code>getStatus(Path p)</code> Returns a status object describing the use and capacity of the file system.
<code>abstract URI</code>	<code>getUri()</code> Returns a URI whose scheme and authority identify this FileSystem.
<code>long</code>	<code>getUsed()</code> Return the total size of all files in the filesystem.
<code>abstract Path</code>	<code>getWorkingDirectory()</code> Get the current working directory for the given file system
<code>byte[]</code>	<code>getXAttr(Path path, String name)</code> Get an xattr name and value for a file or directory.
<code>Map<String,byte[]></code>	<code>getXAttrs(Path path)</code> Get all of the xattr name/value pairs for a file or directory.
<code>Map<String,byte[]></code>	<code>getXAttrs(Path path, List<String> names)</code> Get all of the xattrs name/value pairs for a file or directory.
<code>FileStatus[]</code>	<code>globStatus(Path pathPattern)</code> Return all the files that match filePattern and are not checksum files.
<code>FileStatus[]</code>	<code>globStatus(Path pathPattern, PathFilter filter)</code> Return an array of FileStatus objects whose path names match pathPattern and is accepted by the user-supplied path filter.
<code>void</code>	<code>initialize(URI name, Configuration conf)</code> Called after a new FileSystem instance is constructed.
<code>boolean</code>	<code>isDirectory(Path f)</code> True iff the named path is a directory.
<code>boolean</code>	<code>isFile(Path f)</code> True iff the named path is a regular file.
<code>org.apache.hadoop.fs.RemoteIterator<Path></code>	<code>listCorruptFileBlocks(Path path)</code>
<code>org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus></code>	<code>listFiles(Path f, boolean recursive)</code> List the statuses and block locations of the files in the given path.

<code>org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus></code>	<code>listLocatedStatus(Path f)</code> List the statuses of the files/directories in the given path if the path is a directory.
<code>protected org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus></code>	<code>listLocatedStatus(Path f, PathFilter filter)</code> Listing a directory The returned results include its block location if it is a file The results are filtered by the given path filter
<code>abstract FileStatus[]</code>	<code>listStatus(Path f)</code> List the statuses of the files/directories in the given path if the path is a directory.
<code>FileStatus[]</code>	<code>listStatus(Path[] files)</code> Filter files/directories in the given list of paths using default path filter.
<code>FileStatus[]</code>	<code>listStatus(Path[] files, PathFilter filter)</code> Filter files/directories in the given list of paths using user-supplied path filter.
<code>FileStatus[]</code>	<code>listStatus(Path f, PathFilter filter)</code> Filter files/directories in the given path using the user-supplied path filter.
<code>org.apache.hadoop.fs.RemoteIterator<FileStatus></code>	<code>listStatusIterator(Path p)</code> Returns a remote iterator so that followup calls are made on demand while consuming the entries.
<code>List<String></code>	<code>listXAttrs(Path path)</code> Get all of the xattr names for a file or directory.
<code>Path</code>	<code>makeQualified(Path path)</code> Make sure that a path specifies a FileSystem.
<code>static boolean</code>	<code>mkdirs(FileSystem fs, Path dir, FsPermission permission)</code> create a directory with the provided permission The permission of the directory is set to be the provided permission as in <code>setPermission</code> , not <code>permission&~umask</code>
<code>boolean</code>	<code>mkdirs(Path f)</code> Call <code>mkdirs(Path, FsPermission)</code> with default permission.
<code>abstract boolean</code>	<code>mkdirs(Path f, FsPermission permission)</code> Make the given file and all non-existent parents into directories.
<code>void</code>	<code>modifyAclEntries(Path path, List<AclEntry> aclSpec)</code> Modifies ACL entries of files and directories.
<code>void</code>	<code>moveFromLocalFile(Path[] srcs, Path dst)</code> The src files is on the local disk.
<code>void</code>	<code>moveFromLocalFile(Path src, Path dst)</code> The src file is on the local disk.
<code>void</code>	<code>moveToLocalFile(Path src, Path dst)</code> The src file is under FS, and the dst is on the local disk.
<code>static FileSystem</code>	<code>newInstance(Configuration conf)</code> Returns a unique configured filesystem implementation.
<code>static FileSystem</code>	<code>newInstance(URI uri, Configuration conf)</code> Returns the FileSystem for this URI's scheme and authority.
<code>static FileSystem</code>	<code>newInstance(URI uri, Configuration conf, String user)</code> Returns the FileSystem for this URI's scheme and authority and the passed user.
<code>static LocalFileSystem</code>	<code>newInstanceLocal(Configuration conf)</code> Get a unique local file system object
<code>FSDaataInputStream</code>	<code>open(Path f)</code> Opens an FSDaataInputStream at the indicated Path.
<code>abstract FSDaataInputStream</code>	<code>open(Path f, int bufferSize)</code> Opens an FSDaataInputStream at the indicated Path.
<code>protected FSDaataOutputStream</code>	<code>primitiveCreate(Path f, FsPermission absolutePermission, EnumSet<CreateFlag> flag, int bufferSize, short replication, long blockSize, Progressable progress, org.apache.hadoop.fs.Options.ChecksumOpt checksumOpt)</code> Deprecated.
<code>protected boolean</code>	<code>primitiveMkdir(Path f, FsPermission absolutePermission)</code> Deprecated.
<code>protected void</code>	<code>primitiveMkdir(Path f, FsPermission absolutePermission, boolean createParent)</code> Deprecated.
<code>static void</code>	<code>printStatistics()</code> Print all statistics for all file systems
<code>protected void</code>	<code>processDeleteOnExit()</code> Delete all files that were marked as delete-on-exit.
<code>void</code>	<code>removeAcl(Path path)</code> Removes all but the base ACL entries of files and directories.
<code>void</code>	<code>removeAclEntries(Path path, List<AclEntry> aclSpec)</code> Removes ACL entries from files and directories.
<code>void</code>	<code>removeDefaultAcl(Path path)</code> Removes all default ACL entries from files and directories.
<code>void</code>	<code>removeXAttr(Path path, String name)</code> Remove an xattr of a file or directory.
<code>abstract boolean</code>	<code>rename(Path src, Path dst)</code> Renames Path src to Path dst.
<code>protected void</code>	<code>rename(Path src, Path dst, org.apache.hadoop.fs.Options.Rename... options)</code>

11/3/2015	FileSystem (Apache Hadoop Main 2.7.1 API)
void	Deprecated. <code>renameSnapshot(Path path, String snapshotOldName, String snapshotNewName)</code> Rename a snapshot
protected Path	<code>resolveLink(Path f)</code> See <code>AbstractFileSystem.getLinkTarget(Path)</code>
Path	<code>resolvePath(Path p)</code> Return the fully-qualified path of path f resolving the path through any symlinks or mount point
void	<code>setAcl(Path path, List<AclEntry> aclSpec)</code> Fully replaces ACL of files and directories, discarding all existing entries.
static void	<code>setDefaultUri(Configuration conf, String uri)</code> Set the default filesystem URI in a configuration.
static void	<code>setDefaultUri(Configuration conf, URI uri)</code> Set the default filesystem URI in a configuration.
void	<code>setOwner(Path p, String username, String groupname)</code> Set owner of a path (i.e.
void	<code>setPermission(Path p, FsPermission permission)</code> Set permission of a path.
boolean	<code>setReplication(Path src, short replication)</code> Set replication for an existing file.
void	<code>setTimes(Path p, long mtime, long atime)</code> Set access time of a file
void	<code>setVerifyChecksum(boolean verifyChecksum)</code> Set the verify checksum flag.
abstract void	<code>setWorkingDirectory(Path new_dir)</code> Set the current working directory for the given file system.
void	<code>setWriteChecksum(boolean writeChecksum)</code> Set the write checksum flag.
void	<code>setXAttr(Path path, String name, byte[] value)</code> Set an xattr of a file or directory.
void	<code>setXAttr(Path path, String name, byte[] value, EnumSet<XAttrSetFlag> flag)</code> Set an xattr of a file or directory.
Path	<code>startLocalOutput(Path fsOutputFile, Path tmpLocalFile)</code> Returns a local File that the user can write output to.
boolean	<code>supportsSymlinks()</code> See <code>AbstractFileSystem.supportsSymlinks()</code>
boolean	<code>truncate(Path f, long newLength)</code> Truncate the file in the indicated path to the indicated size.

Methods inherited from class org.apache.hadoop.conf.Configured

getConf, setConf

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

FS_DEFAULT_NAME_KEY

public static final String FS_DEFAULT_NAME_KEY

See Also:

Constant Field Values

DEFAULT_FS

public static final String DEFAULT_FS

See Also:

Constant Field Values

LOG

public static final org.apache.commons.logging.Log LOG

SHUTDOWN_HOOK_PRIORITY

public static final int SHUTDOWN_HOOK_PRIORITY

Priority of the FileSystem shutdown hook.

See Also:

Constant Field Values

statistics

protected org.apache.hadoop.fs.FileSystem.Statistics statistics

The statistics for this file system.

Constructor Detail

FileSystem

protected FileSystem()

Method Detail

get

```
public static FileSystem get(URI uri,
                             Configuration conf,
                             String user)
                             throws IOException,
                             InterruptedException
```

Get a filesystem instance based on the uri, the passed configuration and the user

Parameters:

uri - Of the filesystem

conf - the configuration to use

user - to perform the get as

Returns:

the filesystem instance

Throws:

IOException

InterruptedException

get

```
public static FileSystem get(Configuration conf)
                             throws IOException
```

Returns the configured filesystem implementation.

Parameters:

conf - the configuration to use

Throws:

IOException

getDefaultUri

```
public static URI getDefaultUri(Configuration conf)
```

Get the default filesystem URI from a configuration.

Parameters:

conf - the configuration to use

Returns:

the uri of the default filesystem

setDefaultUri

```
public static void setDefaultUri(Configuration conf,
                                URI uri)
```

Set the default filesystem URI in a configuration.

Parameters:

conf - the configuration to alter
uri - the new default filesystem uri

setDefaultUri

```
public static void setDefaultUri(Configuration conf,
                                String uri)
```

Set the default filesystem URI in a configuration.

Parameters:

conf - the configuration to alter
uri - the new default filesystem uri

initialize

```
public void initialize(URI name,
                      Configuration conf)
    throws IOException
```

Called after a new FileSystem instance is constructed.

Parameters:

name - a uri whose authority section names the host, port, etc. for this FileSystem
conf - the configuration

Throws:

IOException

getScheme

```
public String getScheme()
```

Return the protocol scheme for the FileSystem.

This implementation throws an UnsupportedOperationException.

Returns:

the protocol scheme for the FileSystem.

getUri

```
public abstract URI getUri()
```

Returns a URI whose scheme and authority identify this FileSystem.

getCanonicalUri

```
protected URI getCanonicalUri()
```

Return a canonicalized form of this FileSystem's URI. The default implementation simply calls `canonicalizeUri(URI)` on the filesystem's own URI, so subclasses typically only need to implement that method.

See Also:

`canonicalizeUri(URI)`

canonicalizeUri

```
protected URI canonicalizeUri(URI uri)
```

Canonicalize the given URI. This is filesystem-dependent, but may for example consist of canonicalizing the hostname using DNS and adding the default port if not specified. The default implementation simply fills in the default port if not specified and if the filesystem has a default port.

Returns:

URI

See Also:

NetUtils.getCanonicalUri(URI, int)

getDefaultPort

```
protected int getDefaultPort()
```

Get the default port for this file system.

Returns:

the default port or 0 if there isn't one

getFSofPath

```
protected static FileSystem getFSofPath(Path absOrFqPath,
                                         Configuration conf)
                                throws UnsupportedOperationException,
                                IOException
```

Throws:

UnsupportedFileSystemException

IOException

getName

```
@Deprecated
public String getName()
```

Deprecated. call #getUri() instead.

getNamed

```
@Deprecated
public static FileSystem getNamed(String name,
                                  Configuration conf)
                                throws IOException
```

Deprecated. call #get(URI,Configuration) instead.

Throws:

IOException

getLocal

```
public static LocalFileSystem getLocal(Configuration conf)
                                throws IOException
```

Get the local file system.

Parameters:

conf - the configuration to configure the file system with

Returns:

a LocalFileSystem

Throws:

IOException

get

```
public static FileSystem get(URI uri,
```

Configuration conf)
throws IOException

Returns the FileSystem for this URI's scheme and authority. The scheme of the URI determines a configuration property name, `fs.scheme.class` whose value names the FileSystem class. The entire URI is passed to the FileSystem instance's initialize method.

Throws:

IOException

newInstance

```
public static FileSystem newInstance(URI uri,
                                   Configuration conf,
                                   String user)
    throws IOException,
    InterruptedException
```

Returns the FileSystem for this URI's scheme and authority and the passed user. Internally invokes `newInstance(URI, Configuration)`

Parameters:

- uri - of the filesystem
- conf - the configuration to use
- user - to perform the get as

Returns:

filesystem instance

Throws:

IOException
InterruptedException

newInstance

```
public static FileSystem newInstance(URI uri,
                                   Configuration conf)
    throws IOException
```

Returns the FileSystem for this URI's scheme and authority. The scheme of the URI determines a configuration property name, `fs.scheme.class` whose value names the FileSystem class. The entire URI is passed to the FileSystem instance's initialize method. This always returns a new FileSystem object.

Throws:

IOException

newInstance

```
public static FileSystem newInstance(Configuration conf)
    throws IOException
```

Returns a unique configured filesystem implementation. This always returns a new FileSystem object.

Parameters:

- conf - the configuration to use

Throws:

IOException

newInstanceLocal

```
public static LocalFileSystem newInstanceLocal(Configuration conf)
    throws IOException
```

Get a unique local file system object

Parameters:

- conf - the configuration to configure the file system with

Returns:

a LocalFileSystem This always returns a new FileSystem object.

Throws:

IOException

closeAll

```
public static void closeAll()
    throws IOException
```

Close all cached filesystems. Be sure those filesystems are not used anymore.

Throws:

IOException

closeAllForUGI

```
public static void closeAllForUGI(org.apache.hadoop.security.UserGroupInformation ugi)
    throws IOException
```

Close all cached filesystems for a given UGI. Be sure those filesystems are not used anymore.

Parameters:

ugi - user group info to close

Throws:

IOException

makeQualified

```
public Path makeQualified(Path path)
```

Make sure that a path specifies a FileSystem.

Parameters:

path - to use

create

```
public static FSDataOutputStream create(FileSystem fs,
    Path file,
    FsPermission permission)
    throws IOException
```

create a file with the provided permission The permission of the file is set to be the provided permission as in setPermission, not permission&~umask It is implemented using two RPCs. It is understood that it is inefficient, but the implementation is thread-safe. The other option is to change the value of umask in configuration to be 0, but it is not thread-safe.

Parameters:

fs - file system handle

file - the name of the file to be created

permission - the permission of the file

Returns:

an output stream

Throws:

IOException

mkdirs

```
public static boolean mkdirs(FileSystem fs,
    Path dir,
    FsPermission permission)
    throws IOException
```

create a directory with the provided permission The permission of the directory is set to be the provided permission as in setPermission, not permission&~umask

Parameters:

fs - file system handle

dir - the name of the directory to be created

permission - the permission of the directory

Returns:

true if the directory creation succeeds; false otherwise

Throws:

`IOException`

See Also:

`create(FileSystem, Path, FsPermission)`

checkPath

protected void `checkPath(Path path)`

Check that a Path belongs to this FileSystem.

Parameters:

path - to check

getFileBlockLocations

```
public BlockLocation[] getFileBlockLocations(FileStatus file,
                                             long start,
                                             long len)
    throws IOException
```

Return an array containing hostnames, offset and size of portions of the given file. For a nonexistent file or regions, null will be returned. This call is most helpful with DFS, where it returns hostnames of machines that contain the given file. The FileSystem will simply return an elt containing 'localhost'.

Parameters:

- file - FilesStatus to get data from
- start - offset into the given file
- len - length for which to get locations for

Throws:

`IOException`

getFileBlockLocations

```
public BlockLocation[] getFileBlockLocations(Path p,
                                             long start,
                                             long len)
    throws IOException
```

Return an array containing hostnames, offset and size of portions of the given file. For a nonexistent file or regions, null will be returned. This call is most helpful with DFS, where it returns hostnames of machines that contain the given file. The FileSystem will simply return an elt containing 'localhost'.

Parameters:

- p - path is used to identify an FS since an FS could have another FS that it could be delegating the call to
- start - offset into the given file
- len - length for which to get locations for

Throws:

`IOException`

getServerDefaults

```
@Deprecated
public FsServerDefaults getServerDefaults()
    throws IOException
```

Deprecated. use `getServerDefaults(Path)` instead
Return a set of server default configuration values

Returns:

server default configuration values

Throws:

`IOException`

getServerDefaults

```
public FsServerDefaults getServerDefaults(Path p)
    throws IOException
```

Return a set of server default configuration values

Parameters:

p - path is used to identify an FS since an FS could have another FS that it could be delegating the call to

Returns:

server default configuration values

Throws:

IOException

resolvePath

```
public Path resolvePath(Path p)
    throws IOException
```

Return the fully-qualified path of path f resolving the path through any symlinks or mount point

Parameters:

p - path to be resolved

Returns:

fully qualified path

Throws:

FileNotFoundException

IOException

open

```
public abstract FSDataInputStream open(Path f,
    int bufferSize)
    throws IOException
```

Opens an FSDataInputStream at the indicated Path.

Parameters:

f - the file name to open

bufferSize - the size of the buffer to be used.

Throws:

IOException

open

```
public FSDataInputStream open(Path f)
    throws IOException
```

Opens an FSDataInputStream at the indicated Path.

Parameters:

f - the file to open

Throws:

IOException

create

```
public FSDataOutputStream create(Path f)
    throws IOException
```

Create an FSDataOutputStream at the indicated Path. Files are overwritten by default.

Parameters:

f - the file to create

Throws:

IOException

create

```
public FSDataOutputStream create(Path f,
    boolean overwrite)
```

throws `IOException`

Create an `FSDDataOutputStream` at the indicated Path.

Parameters:

f - the file to create

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an exception will be thrown.

Throws:

`IOException`

create

```
public FSDDataOutputStream create(Path f,
                                   Progressable progress)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path with write-progress reporting. Files are overwritten by default.

Parameters:

f - the file to create

progress - to report progress

Throws:

`IOException`

create

```
public FSDDataOutputStream create(Path f,
                                   short replication)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path. Files are overwritten by default.

Parameters:

f - the file to create

replication - the replication factor

Throws:

`IOException`

create

```
public FSDDataOutputStream create(Path f,
                                   short replication,
                                   Progressable progress)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path with write-progress reporting. Files are overwritten by default.

Parameters:

f - the file to create

replication - the replication factor

progress - to report progress

Throws:

`IOException`

create

```
public FSDDataOutputStream create(Path f,
                                   boolean overwrite,
                                   int bufferSize)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path.

Parameters:

f - the file name to create

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

bufferSize - the size of the buffer to be used.

Throws:`IOException`**create**

```
public FSDataOutputStream create(Path f,
                                boolean overwrite,
                                int bufferSize,
                                Progressable progress)
    throws IOException
```

Create an FSDataOutputStream at the indicated Path with write-progress reporting.

Parameters:

f - the path of the file to open

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

bufferSize - the size of the buffer to be used.

Throws:`IOException`**create**

```
public FSDataOutputStream create(Path f,
                                boolean overwrite,
                                int bufferSize,
                                short replication,
                                long blockSize)
    throws IOException
```

Create an FSDataOutputStream at the indicated Path.

Parameters:

f - the file name to open

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

bufferSize - the size of the buffer to be used.

replication - required block replication for the file.

Throws:`IOException`**create**

```
public FSDataOutputStream create(Path f,
                                boolean overwrite,
                                int bufferSize,
                                short replication,
                                long blockSize,
                                Progressable progress)
    throws IOException
```

Create an FSDataOutputStream at the indicated Path with write-progress reporting.

Parameters:

f - the file name to open

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

bufferSize - the size of the buffer to be used.

replication - required block replication for the file.

Throws:`IOException`**create**

```
public abstract FSDataOutputStream create(Path f,
                                           FsPermission permission,
                                           boolean overwrite,
                                           int bufferSize,
                                           short replication,
                                           long blockSize,
                                           Progressable progress)
```

throws `IOException`

Create an `FSDDataOutputStream` at the indicated Path with write-progress reporting.

Parameters:

`f` - the file name to open

`permission` -

`overwrite` - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

`bufferSize` - the size of the buffer to be used.

`replication` - required block replication for the file.

`blockSize` -

`progress` -

Throws:

`IOException`

See Also:

`setPermission(Path, FsPermission)`

create

```
public FSDDataOutputStream create(Path f,
                                   FsPermission permission,
                                   EnumSet<CreateFlag> flags,
                                   int bufferSize,
                                   short replication,
                                   long blockSize,
                                   Progressable progress)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path with write-progress reporting.

Parameters:

`f` - the file name to open

`permission` -

`flags` - `CreateFlags` to use for this stream.

`bufferSize` - the size of the buffer to be used.

`replication` - required block replication for the file.

`blockSize` -

`progress` -

Throws:

`IOException`

See Also:

`setPermission(Path, FsPermission)`

create

```
public FSDDataOutputStream create(Path f,
                                   FsPermission permission,
                                   EnumSet<CreateFlag> flags,
                                   int bufferSize,
                                   short replication,
                                   long blockSize,
                                   Progressable progress,
                                   org.apache.hadoop.fs.Options.ChecksumOpt checksumOpt)
    throws IOException
```

Create an `FSDDataOutputStream` at the indicated Path with a custom checksum option

Parameters:

`f` - the file name to open

`permission` -

`flags` - `CreateFlags` to use for this stream.

`bufferSize` - the size of the buffer to be used.

`replication` - required block replication for the file.

`blockSize` -

`progress` -

`checksumOpt` - checksum parameter. If null, the values found in conf will be used.

Throws:

IOException

See Also:

setPermission(Path, FsPermission)

primitiveCreate

```
@Deprecated
protected FSDataOutputStream primitiveCreate(Path f,
                                             FsPermission absolutePermission,
                                             EnumSet<CreateFlag> flag,
                                             int bufferSize,
                                             short replication,
                                             long blockSize,
                                             Progressable progress,
                                             org.apache.hadoop.fs.Options.ChecksumOpt checksumOpt)
    throws IOException
```

Deprecated.

Throws:

IOException

primitiveMkdir

```
@Deprecated
protected boolean primitiveMkdir(Path f,
                                 FsPermission absolutePermission)
    throws IOException
```

Deprecated.

This version of the mkdirs method assumes that the permission is absolute. It has been added to support the FileContext that processes the permission with umask before calling this method. This a temporary method added to support the transition from FileSystem to FileContext for user applications.

Throws:

IOException

primitiveMkdir

```
@Deprecated
protected void primitiveMkdir(Path f,
                              FsPermission absolutePermission,
                              boolean createParent)
    throws IOException
```

Deprecated.

This version of the mkdirs method assumes that the permission is absolute. It has been added to support the FileContext that processes the permission with umask before calling this method. This a temporary method added to support the transition from FileSystem to FileContext for user applications.

Throws:

IOException

createNonRecursive

```
@Deprecated
public FSDataOutputStream createNonRecursive(Path f,
                                             boolean overwrite,
                                             int bufferSize,
                                             short replication,
                                             long blockSize,
                                             Progressable progress)
    throws IOException
```

Deprecated. API only for 0.20-append

Opens an FSDataOutputStream at the indicated Path with write-progress reporting. Same as create(), except fails if parent directory doesn't already exist.

Parameters:

- f - the file name to open
- overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.
- bufferSize - the size of the buffer to be used.
- replication - required block replication for the file.
- blockSize -

progress -

Throws:

`IOException`

See Also:

`setPermission(Path, FsPermission)`

createNonRecursive

`@Deprecated`

```
public FSDataOutputStream createNonRecursive(Path f,
                                             FsPermission permission,
                                             boolean overwrite,
                                             int bufferSize,
                                             short replication,
                                             long blockSize,
                                             Progressable progress)
    throws IOException
```

Deprecated. *API only for 0.20-append*

Opens an FSDataOutputStream at the indicated Path with write-progress reporting. Same as create(), except fails if parent directory doesn't already exist.

Parameters:

f - the file name to open

permission -

overwrite - if a file with this name already exists, then if true, the file will be overwritten, and if false an error will be thrown.

bufferSize - the size of the buffer to be used.

replication - required block replication for the file.

blockSize -

progress -

Throws:

`IOException`

See Also:

`setPermission(Path, FsPermission)`

createNonRecursive

`@Deprecated`

```
public FSDataOutputStream createNonRecursive(Path f,
                                             FsPermission permission,
                                             EnumSet<CreateFlag> flags,
                                             int bufferSize,
                                             short replication,
                                             long blockSize,
                                             Progressable progress)
    throws IOException
```

Deprecated. *API only for 0.20-append*

Opens an FSDataOutputStream at the indicated Path with write-progress reporting. Same as create(), except fails if parent directory doesn't already exist.

Parameters:

f - the file name to open

permission -

flags - `CreateFlags` to use for this stream.

bufferSize - the size of the buffer to be used.

replication - required block replication for the file.

blockSize -

progress -

Throws:

`IOException`

See Also:

`setPermission(Path, FsPermission)`

createNewFile

```
public boolean createNewFile(Path f)
```

throws `IOException`

Creates the given Path as a brand-new zero-length file. If create fails, or if it already existed, return false.

Parameters:

f - path to use for create

Throws:

`IOException`

append

```
public FSDataOutputStream append(Path f)
    throws IOException
```

Append to an existing file (optional operation). Same as `append(f, getConf().getInt("io.file.buffer.size", 4096), null)`

Parameters:

f - the existing file to be appended.

Throws:

`IOException`

append

```
public FSDataOutputStream append(Path f,
    int bufferSize)
    throws IOException
```

Append to an existing file (optional operation). Same as `append(f, bufferSize, null)`.

Parameters:

f - the existing file to be appended.

bufferSize - the size of the buffer to be used.

Throws:

`IOException`

append

```
public abstract FSDataOutputStream append(Path f,
    int bufferSize,
    Progressable progress)
    throws IOException
```

Append to an existing file (optional operation).

Parameters:

f - the existing file to be appended.

bufferSize - the size of the buffer to be used.

progress - for reporting progress if it is not null.

Throws:

`IOException`

concat

```
public void concat(Path trg,
    Path[] psrcs)
    throws IOException
```

Concat existing files together.

Parameters:

trg - the path to the target destination.

psrcs - the paths to the sources to use for the concatenation.

Throws:

`IOException`

getReplication

```
@Deprecated
public short getReplication(Path src)
    throws IOException
```

Deprecated. Use *getFileStatus()* instead
Get replication.

Parameters:
src - file name

Returns:
file replication

Throws:
IOException

setReplication

```
public boolean setReplication(Path src,
    short replication)
    throws IOException
```

Set replication for an existing file.

Parameters:
src - file name
replication - new replication

Returns:
true if successful; false if file does not exist or is a directory

Throws:
IOException

rename

```
public abstract boolean rename(Path src,
    Path dst)
    throws IOException
```

Renames Path src to Path dst. Can take place on local fs or remote DFS.

Parameters:
src - path to be renamed
dst - new path after rename

Returns:
true if rename is successful

Throws:
IOException - on failure

rename

```
@Deprecated
protected void rename(Path src,
    Path dst,
    org.apache.hadoop.fs.Options.Rename... options)
    throws IOException
```

Deprecated.
Renames Path src to Path dst

- Fails if src is a file and dst is a directory.
- Fails if src is a directory and dst is a file.
- Fails if the parent of dst does not exist or is a file.

If OVERWRITE option is not passed as an argument, rename fails if the dst already exists.

If OVERWRITE option is passed as an argument, rename overwrites the dst if it is a file or an empty directory. Rename fails if dst is a non-empty directory.

Note that atomicity of rename is dependent on the file system implementation. Please refer to the file system documentation for details. This default implementation is non atomic.

This method is deprecated since it is a temporary method added to support the transition from FileSystem to FileContext for user applications.

Parameters:
src - path to be renamed

dst - new path after rename

Throws:

IOException - on failure

truncate

```
public boolean truncate(Path f,
                        long newLength)
    throws IOException
```

Truncate the file in the indicated path to the indicated size.

- Fails if path is a directory.
- Fails if path does not exist.
- Fails if path is not closed.
- Fails if new size is greater than current size.

Parameters:

f - The path to the file to be truncated

newLength - The size the file is to be truncated to

Returns:

true if the file has been truncated to the desired newLength and is immediately available to be reused for write operations such as append, or false if a background process of adjusting the length of the last block has been started, and clients should wait for it to complete before proceeding with further file updates.

Throws:

IOException

delete

```
@Deprecated
public boolean delete(Path f)
    throws IOException
```

Deprecated. Use `delete(Path, boolean)` instead.
Delete a file

Throws:

IOException

delete

```
public abstract boolean delete(Path f,
                               boolean recursive)
    throws IOException
```

Delete a file.

Parameters:

f - the path to delete.

recursive - if path is a directory and set to true, the directory is deleted else throws an exception. In case of a file the recursive can be set to either true or false.

Returns:

true if delete is successful else false.

Throws:

IOException

deleteOnExit

```
public boolean deleteOnExit(Path f)
    throws IOException
```

Mark a path to be deleted when FileSystem is closed. When the JVM shuts down, all FileSystem objects will be closed automatically. Then, the marked path will be deleted as a result of closing the FileSystem. The path has to exist in the file system.

Parameters:

f - the path to delete.

Returns:

true if deleteOnExit is successful, otherwise false.

Throws:

`IOException`

cancelDeleteOnExit

```
public boolean cancelDeleteOnExit(Path f)
```

Cancel the deletion of the path when the FileSystem is closed

Parameters:

f - the path to cancel deletion

processDeleteOnExit

```
protected void processDeleteOnExit()
```

Delete all files that were marked as delete-on-exit. This recursively deletes all files in the specified paths.

exists

```
public boolean exists(Path f)
    throws IOException
```

Check if exists.

Parameters:

f - source file

Throws:

`IOException`

isDirectory

```
public boolean isDirectory(Path f)
    throws IOException
```

True iff the named path is a directory. Note: Avoid using this method. Instead reuse the `FileStatus` returned by `getFileStatus()` or `listStatus()` methods.

Parameters:

f - path to check

Throws:

`IOException`

isFile

```
public boolean isFile(Path f)
    throws IOException
```

True iff the named path is a regular file. Note: Avoid using this method. Instead reuse the `FileStatus` returned by `getFileStatus()` or `listStatus()` methods.

Parameters:

f - path to check

Throws:

`IOException`

getLength

```
@Deprecated
public long getLength(Path f)
    throws IOException
```

Deprecated. Use `getFileStatus()` instead

Throws:

`IOException`

getContentSummary

```
public ContentSummary getContentSummary(Path f)
```

throws `IOException`

Return the `ContentSummary` of a given `Path`.

Parameters:

f - path to use

Throws:

`IOException`

listStatus

```
public abstract FileStatus[] listStatus(Path f)
                                throws FileNotFoundException,
                                IOException
```

List the statuses of the files/directories in the given path if the path is a directory.

Parameters:

f - given path

Returns:

the statuses of the files/directories in the given patch

Throws:

`FileNotFoundException` - when the path does not exist; `IOException` see specific implementation

`IOException`

listCorruptFileBlocks

```
public org.apache.hadoop.fs.RemoteIterator<Path> listCorruptFileBlocks(Path path)
                                                                    throws IOException
```

Returns:

an iterator over the corrupt files under the given path (may contain duplicates if a file has more than one corrupt block)

Throws:

`IOException`

listStatus

```
public FileStatus[] listStatus(Path f,
                                PathFilter filter)
                                throws FileNotFoundException,
                                IOException
```

Filter files/directories in the given path using the user-supplied path filter.

Parameters:

f - a path name

filter - the user-supplied path filter

Returns:

an array of `FileStatus` objects for the files under the given path after applying the filter

Throws:

`FileNotFoundException` - when the path does not exist; `IOException` see specific implementation

`IOException`

listStatus

```
public FileStatus[] listStatus(Path[] files)
                                throws FileNotFoundException,
                                IOException
```

Filter files/directories in the given list of paths using default path filter.

Parameters:

files - a list of paths

Returns:

a list of statuses for the files under the given paths after applying the filter default `Path` filter

Throws:

`FileNotFoundException` - when the path does not exist; `IOException` see specific implementation

`IOException`

listStatus

```
public FileStatus[] listStatus(Path[] files,
                               PathFilter filter)
    throws FileNotFoundException,
    IOException
```

Filter files/directories in the given list of paths using user-supplied path filter.

Parameters:

- `files` - a list of paths
- `filter` - the user-supplied path filter

Returns:

a list of statuses for the files under the given paths after applying the filter

Throws:

`FileNotFoundException` - when the path does not exist; `IOException` see specific implementation

`IOException`

globStatus

```
public FileStatus[] globStatus(Path pathPattern)
    throws IOException
```

Return all the files that match filePattern and are not checksum files. Results are sorted by their names.

A filename pattern is composed of *regular* characters and *special pattern matching* characters, which are:

- `?`
Matches any single character.
- `*`
Matches zero or more characters.
- `[abc]`
Matches a single character from character set {*a*, *b*, *c*}.
- `[a-b]`
Matches a single character from the character range {*a* . . *b*}. Note that character *a* must be lexicographically less than or equal to character *b*.
- `[^a]`
Matches a single character that is not from character set or range {*a*}. Note that the ^ character must occur immediately to the right of the opening bracket.
- `\c`
Removes (escapes) any special meaning of character *c*.
- `{ab,cd}`
Matches a string from the string set {*ab*, *cd*}
- `{ab,c{de,fh}}`
Matches a string from the string set {*ab*, *cde*, *cfh*}

Parameters:

- `pathPattern` - a regular expression specifying a path pattern

Returns:

an array of paths that match the path pattern

Throws:

`IOException`

globStatus

```
public FileStatus[] globStatus(Path pathPattern,
                               PathFilter filter)
    throws IOException
```

Return an array of FileStatus objects whose path names match pathPattern and is accepted by the user-supplied path filter. Results are sorted by their path names. Return null if pathPattern has no glob and the path does not exist. Return an empty array if pathPattern has a glob and no path matches it.

Parameters:

- pathPattern - a regular expression specifying the path pattern
- filter - a user-supplied path filter

Returns:

- an array of FileStatus objects

Throws:

- [IOException](#) - if any I/O error occurs when fetching file status

listLocatedStatus

```
public org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus> listLocatedStatus(Path f)
                                                                    throws FileNotFoundException,
                                                                    IOException
```

List the statuses of the files/directories in the given path if the path is a directory. Return the file's status and block locations If the path is a file. If a returned status is a file, it contains the file's block locations.

Parameters:

- f - is the path

Returns:

- an iterator that traverses statuses of the files/directories in the given path

Throws:

- [FileNotFoundException](#) - If f does not exist
- [IOException](#) - If an I/O error occurred

listLocatedStatus

```
protected org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus> listLocatedStatus(Path f,
                                                                    PathFilter filter)
                                                                    throws FileNotFoundException,
                                                                    IOException
```

Listing a directory The returned results include its block location if it is a file The results are filtered by the given path filter

Parameters:

- f - a path
- filter - a path filter

Returns:

- an iterator that traverses statuses of the files/directories in the given path

Throws:

- [FileNotFoundException](#) - if f does not exist
- [IOException](#) - if any I/O error occurred

listStatusIterator

```
public org.apache.hadoop.fs.RemoteIterator<FileStatus> listStatusIterator(Path p)
                                                                    throws FileNotFoundException,
                                                                    IOException
```

Returns a remote iterator so that followup calls are made on demand while consuming the entries. Each file system implementation should override this method and provide a more efficient implementation, if possible.

Parameters:

- p - target path

Returns:

- remote iterator

Throws:

- [FileNotFoundException](#)
- [IOException](#)

listFiles

```
public org.apache.hadoop.fs.RemoteIterator<LocatedFileStatus> listFiles(Path f,
                                                                    boolean recursive)
```

throws `FileNotFoundException`,
`IOException`

List the statuses and block locations of the files in the given path. If the path is a directory, if recursive is false, returns files in the directory; if recursive is true, return files in the subtree rooted at the path. If the path is a file, return the file's status and block locations.

Parameters:

- f - is the path
- recursive - if the subdirectories need to be traversed recursively

Returns:

an iterator that traverses statuses of the files

Throws:

- `FileNotFoundException` - when the path does not exist; `IOException` see specific implementation
- `IOException`

getHomeDirectory

```
public Path getHomeDirectory()
```

Return the current user's home directory in this filesystem. The default implementation returns "/user/\$USER/".

setWorkingDirectory

```
public abstract void setWorkingDirectory(Path new_dir)
```

Set the current working directory for the given file system. All relative paths will be resolved relative to it.

Parameters:

- new_dir -

getWorkingDirectory

```
public abstract Path getWorkingDirectory()
```

Get the current working directory for the given file system

Returns:

the directory pathname

getInitialWorkingDirectory

```
protected Path getInitialWorkingDirectory()
```

Note: with the new `FilesContext` class, `getWorkingDirectory()` will be removed. The working directory is implemented in `FilesContext`. Some file systems like `LocalFileSystem` have an initial `workingDir` that we use as the starting `workingDir`. For other file systems like HDFS there is no built in notion of an initial `workingDir`.

Returns:

if there is built in notion of `workingDir` then it is returned; else a null is returned.

mkdirs

```
public boolean mkdirs(Path f)
    throws IOException
```

Call `mkdirs(Path, FsPermission)` with default permission.

Throws:

- `IOException`

mkdirs

```
public abstract boolean mkdirs(Path f,
    FsPermission permission)
    throws IOException
```

Make the given file and all non-existent parents into directories. Has the semantics of Unix 'mkdir -p'. Existence of the directory hierarchy is not an error.

Parameters:

- f - path to create

permission - to apply to f

Throws:

IOException

copyFromLocalFile

```
public void copyFromLocalFile(Path src,
                               Path dst)
    throws IOException
```

The src file is on the local disk. Add it to FS at the given dst name and the source is kept intact afterwards

Parameters:

- src - path
- dst - path

Throws:

IOException

moveFromLocalFile

```
public void moveFromLocalFile(Path[] srcs,
                               Path dst)
    throws IOException
```

The src files is on the local disk. Add it to FS at the given dst name, removing the source afterwards.

Parameters:

- srcs - path
- dst - path

Throws:

IOException

moveFromLocalFile

```
public void moveFromLocalFile(Path src,
                               Path dst)
    throws IOException
```

The src file is on the local disk. Add it to FS at the given dst name, removing the source afterwards.

Parameters:

- src - path
- dst - path

Throws:

IOException

copyFromLocalFile

```
public void copyFromLocalFile(boolean delSrc,
                               Path src,
                               Path dst)
    throws IOException
```

The src file is on the local disk. Add it to FS at the given dst name. delSrc indicates if the source should be removed

Parameters:

- delSrc - whether to delete the src
- src - path
- dst - path

Throws:

IOException

copyFromLocalFile

```
public void copyFromLocalFile(boolean delSrc,
                               boolean overwrite,
```

```
Path[] srcs,  
Path dst)  
throws IOException
```

The src files are on the local disk. Add it to FS at the given dst name. delSrc indicates if the source should be removed

Parameters:

delSrc - whether to delete the src
overwrite - whether to overwrite an existing file
srcs - array of paths which are source
dst - path

Throws:

IOException

copyFromLocalFile

```
public void copyFromLocalFile(boolean delSrc,  
boolean overwrite,  
Path src,  
Path dst)  
throws IOException
```

The src file is on the local disk. Add it to FS at the given dst name. delSrc indicates if the source should be removed

Parameters:

delSrc - whether to delete the src
overwrite - whether to overwrite an existing file
src - path
dst - path

Throws:

IOException

copyToLocalFile

```
public void copyToLocalFile(Path src,  
Path dst)  
throws IOException
```

The src file is under FS, and the dst is on the local disk. Copy it from FS control to the local dst name.

Parameters:

src - path
dst - path

Throws:

IOException

moveToLocalFile

```
public void moveToLocalFile(Path src,  
Path dst)  
throws IOException
```

The src file is under FS, and the dst is on the local disk. Copy it from FS control to the local dst name. Remove the source afterwards

Parameters:

src - path
dst - path

Throws:

IOException

copyToLocalFile

```
public void copyToLocalFile(boolean delSrc,  
Path src,  
Path dst)  
throws IOException
```

The src file is under FS, and the dst is on the local disk. Copy it from FS control to the local dst name. delSrc indicates if the src will be removed or not.

Parameters:

delSrc - whether to delete the src

src - path

dst - path

Throws:

`IOException`

copyToLocalFile

```
public void copyToLocalFile(boolean delSrc,
    Path src,
    Path dst,
    boolean useRawLocalFileSystem)
    throws IOException
```

The src file is under FS, and the dst is on the local disk. Copy it from FS control to the local dst name. delSrc indicates if the src will be removed or not. useRawLocalFileSystem indicates whether to use RawLocalFileSystem as local file system or not. RawLocalFileSystem is non crc file system. So, It will not create any crc files at local.

Parameters:

delSrc - whether to delete the src

src - path

dst - path

useRawLocalFileSystem - whether to use RawLocalFileSystem as local file system or not.

Throws:

`IOException` -- if any IO error

startLocalOutput

```
public Path startLocalOutput(Path fsOutputFile,
    Path tmpLocalFile)
    throws IOException
```

Returns a local File that the user can write output to. The caller provides both the eventual FS target name and the local working file. If the FS is local, we write directly into the target. If the FS is remote, we write into the tmp local area.

Parameters:

fsOutputFile - path of output file

tmpLocalFile - path of local tmp file

Throws:

`IOException`

completeLocalOutput

```
public void completeLocalOutput(Path fsOutputFile,
    Path tmpLocalFile)
    throws IOException
```

Called when we're all done writing to the target. A local FS will do nothing, because we've written to exactly the right place. A remote FS will copy the contents of tmpLocalFile to the correct target at fsOutputFile.

Parameters:

fsOutputFile - path of output file

tmpLocalFile - path to local tmp file

Throws:

`IOException`

close

```
public void close()
    throws IOException
```

No more filesystem operations are needed. Will release any held locks.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Throws:

`IOException`

getUsed

```
public long getUsed()
           throws IOException
```

Return the total size of all files in the filesystem.

Throws:

`IOException`

getBlockSize

```
@Deprecated
public long getBlockSize(Path f)
           throws IOException
```

Deprecated. Use `getFileStatus()` instead

Throws:

`IOException`

getDefaultBlockSize

```
@Deprecated
public long getDefaultBlockSize()
```

Deprecated. use `getDefaultBlockSize(Path)` instead

Return the number of bytes that large input files should be optimally be split into to minimize i/o time.

getDefaultBlockSize

```
public long getDefaultBlockSize(Path f)
```

Return the number of bytes that large input files should be optimally be split into to minimize i/o time. The given path will be used to locate the actual filesystem. The full path does not have to exist.

Parameters:

`f` - path of file

Returns:

the default block size for the path's filesystem

getDefaultReplication

```
@Deprecated
public short getDefaultReplication()
```

Deprecated. use `getDefaultReplication(Path)` instead

Get the default replication.

getDefaultReplication

```
public short getDefaultReplication(Path path)
```

Get the default replication for a path. The given path will be used to locate the actual filesystem. The full path does not have to exist.

Parameters:

`path` - of the file

Returns:

default replication for the path's filesystem

getFileStatus

```
public abstract FileStatus getFileStatus(Path f)
           throws IOException
```

Return a file status object that represents the path.

Parameters:

f - The path we want information from

Returns:

a FileStatus object

Throws:

FileNotFoundException - when the path does not exist; IOException see specific implementation
IOException

fixRelativePart

protected Path fixRelativePart(Path p)

See FileContext.fixRelativePart(org.apache.hadoop.fs.Path)

createSymlink

```
public void createSymlink(Path target,
                          Path link,
                          boolean createParent)
    throws org.apache.hadoop.security.AccessControlException,
           FileAlreadyExistsException,
           FileNotFoundException,
           ParentNotDirectoryException,
           UnsupportedFileSystemException,
           IOException
```

See FileContext.createSymlink(Path, Path, boolean)

Throws:

org.apache.hadoop.security.AccessControlException
FileAlreadyExistsException
FileNotFoundException
ParentNotDirectoryException
UnsupportedFileSystemException
IOException

getFileLinkStatus

```
public FileStatus getFileLinkStatus(Path f)
    throws org.apache.hadoop.security.AccessControlException,
           FileNotFoundException,
           UnsupportedFileSystemException,
           IOException
```

See FileContext.getFileLinkStatus(Path)

Throws:

org.apache.hadoop.security.AccessControlException
FileNotFoundException
UnsupportedFileSystemException
IOException

supportsSymlinks

public boolean supportsSymlinks()

See AbstractFileSystem.supportsSymlinks()

getLinkTarget

```
public Path getLinkTarget(Path f)
    throws IOException
```

See FileContext.getLinkTarget(Path)

Throws:

IOException

resolveLink

```
protected Path resolveLink(Path f)
    throws IOException
```

See `AbstractFileSystem.getLinkTarget(Path)`

Throws:

`IOException`

getFileChecksum

```
public FileChecksum getFileChecksum(Path f)
    throws IOException
```

Get the checksum of a file.

Parameters:

`f` - The file path

Returns:

The file checksum. The default return value is null, which indicates that no checksum algorithm is implemented in the corresponding `FileSystem`.

Throws:

`IOException`

getFileChecksum

```
public FileChecksum getFileChecksum(Path f,
    long length)
    throws IOException
```

Get the checksum of a file, from the beginning of the file till the specific length.

Parameters:

`f` - The file path

`length` - The length of the file range for checksum calculation

Returns:

The file checksum.

Throws:

`IOException`

setVerifyChecksum

```
public void setVerifyChecksum(boolean verifyChecksum)
```

Set the verify checksum flag. This is only applicable if the corresponding `FileSystem` supports checksum. By default doesn't do anything.

Parameters:

`verifyChecksum` -

setWriteChecksum

```
public void setWriteChecksum(boolean writeChecksum)
```

Set the write checksum flag. This is only applicable if the corresponding `FileSystem` supports checksum. By default doesn't do anything.

Parameters:

`writeChecksum` -

getStatus

```
public FsStatus getStatus()
    throws IOException
```

Returns a status object describing the use and capacity of the file system. If the file system has multiple partitions, the use and capacity of the root partition is reflected.

Returns:

a FsStatus object

Throws:

[IOException](#) - see specific implementation

getStatus

```
public FsStatus getStatus(Path p)
    throws IOException
```

Returns a status object describing the use and capacity of the file system. If the file system has multiple partitions, the use and capacity of the partition pointed to by the specified path is reflected.

Parameters:

p - Path for which status should be obtained. null means the default partition.

Returns:

a FsStatus object

Throws:

[IOException](#) - see specific implementation

setPermission

```
public void setPermission(Path p,
    FsPermission permission)
    throws IOException
```

Set permission of a path.

Parameters:

p -
permission -

Throws:

[IOException](#)

setOwner

```
public void setOwner(Path p,
    String username,
    String groupname)
    throws IOException
```

Set owner of a path (i.e. a file or a directory). The parameters username and groupname cannot both be null.

Parameters:

p - The path
username - If it is null, the original username remains unchanged.
groupname - If it is null, the original groupname remains unchanged.

Throws:

[IOException](#)

setTimes

```
public void setTimes(Path p,
    long mtime,
    long atime)
    throws IOException
```

Set access time of a file

Parameters:

p - The path
mtime - Set the modification time of this file. The number of milliseconds since Jan 1, 1970. A value of -1 means that this call should not set modification time.
atime - Set the access time of this file. The number of milliseconds since Jan 1, 1970. A value of -1 means that this call should not set access time.

Throws:

[IOException](#)

createSnapshot

```
public final Path createSnapshot(Path path)
    throws IOException
```

Create a snapshot with a default name.

Parameters:

path - The directory where snapshots will be taken.

Returns:

the snapshot path.

Throws:

IOException

createSnapshot

```
public Path createSnapshot(Path path,
    String snapshotName)
    throws IOException
```

Create a snapshot

Parameters:

path - The directory where snapshots will be taken.

snapshotName - The name of the snapshot

Returns:

the snapshot path.

Throws:

IOException

renameSnapshot

```
public void renameSnapshot(Path path,
    String snapshotOldName,
    String snapshotNewName)
    throws IOException
```

Rename a snapshot

Parameters:

path - The directory path where the snapshot was taken

snapshotOldName - Old name of the snapshot

snapshotNewName - New name of the snapshot

Throws:

IOException

deleteSnapshot

```
public void deleteSnapshot(Path path,
    String snapshotName)
    throws IOException
```

Delete a snapshot of a directory

Parameters:

path - The directory that the to-be-deleted snapshot belongs to

snapshotName - The name of the snapshot

Throws:

IOException

modifyAclEntries

```
public void modifyAclEntries(Path path,
    List<AclEntry> aclSpec)
    throws IOException
```

Modifies ACL entries of files and directories. This method can add new ACL entries or modify the permissions on existing ACL entries. All existing ACL

entries that are not specified in this call are retained without changes. (Modifications are merged into the current ACL.)

Parameters:

path - Path to modify

aclSpec - List describing modifications

Throws:

[IOException](#) - if an ACL could not be modified

removeAclEntries

```
public void removeAclEntries(Path path,  
                             List<AclEntry> aclSpec)  
    throws IOException
```

Removes ACL entries from files and directories. Other ACL entries are retained.

Parameters:

path - Path to modify

aclSpec - List describing entries to remove

Throws:

[IOException](#) - if an ACL could not be modified

removeDefaultAcl

```
public void removeDefaultAcl(Path path)  
    throws IOException
```

Removes all default ACL entries from files and directories.

Parameters:

path - Path to modify

Throws:

[IOException](#) - if an ACL could not be modified

removeAcl

```
public void removeAcl(Path path)  
    throws IOException
```

Removes all but the base ACL entries of files and directories. The entries for user, group, and others are retained for compatibility with permission bits.

Parameters:

path - Path to modify

Throws:

[IOException](#) - if an ACL could not be removed

setAcl

```
public void setAcl(Path path,  
                   List<AclEntry> aclSpec)  
    throws IOException
```

Fully replaces ACL of files and directories, discarding all existing entries.

Parameters:

path - Path to modify

aclSpec - List describing modifications, must include entries for user, group, and others for compatibility with permission bits.

Throws:

[IOException](#) - if an ACL could not be modified

getAclStatus

```
public AclStatus getAclStatus(Path path)  
    throws IOException
```

Gets the ACL of a file or directory.

Parameters:

path - Path to get

Returns:

AclStatus describing the ACL of the file or directory

Throws:

[IOException](#) - if an ACL could not be read

setXAttr

```
public void setXAttr(Path path,
                    String name,
                    byte[] value)
    throws IOException
```

Set an xattr of a file or directory. The name must be prefixed with the namespace followed by ".". For example, "user.attr".

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to modify

name - xattr name.

value - xattr value.

Throws:

[IOException](#)

setXAttr

```
public void setXAttr(Path path,
                    String name,
                    byte[] value,
                    EnumSet<XAttrSetFlag> flag)
    throws IOException
```

Set an xattr of a file or directory. The name must be prefixed with the namespace followed by ".". For example, "user.attr".

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to modify

name - xattr name.

value - xattr value.

flag - xattr set flag

Throws:

[IOException](#)

getXAttr

```
public byte[] getXAttr(Path path,
                      String name)
    throws IOException
```

Get an xattr name and value for a file or directory. The name must be prefixed with the namespace followed by ".". For example, "user.attr".

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to get extended attribute

name - xattr name.

Returns:

byte[] xattr value.

Throws:

[IOException](#)

getXAttrs

```
public Map<String,byte[]> getXAttrs(Path path)
    throws IOException
```

Get all of the xattr name/value pairs for a file or directory. Only those xattrs which the logged-in user has permissions to view are returned.

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to get extended attributes

Returns:

Map describing the XAttrs of the file or directory

Throws:

`IOException`

getXAttrs

```
public Map<String,byte[]> getXAttrs(Path path,
                                     List<String> names)
    throws IOException
```

Get all of the xattrs name/value pairs for a file or directory. Only those xattrs which the logged-in user has permissions to view are returned.

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to get extended attributes

names - XAttr names.

Returns:

Map describing the XAttrs of the file or directory

Throws:

`IOException`

listXAttrs

```
public List<String> listXAttrs(Path path)
    throws IOException
```

Get all of the xattr names for a file or directory. Only those xattr names which the logged-in user has permissions to view are returned.

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to get extended attributes

Returns:

List of the XAttr names of the file or directory

Throws:

`IOException`

removeXAttr

```
public void removeXAttr(Path path,
                        String name)
    throws IOException
```

Remove an xattr of a file or directory. The name must be prefixed with the namespace followed by ".". For example, "user.attr".

Refer to the HDFS extended attributes user documentation for details.

Parameters:

path - Path to remove extended attribute

name - xattr name

Throws:

`IOException`

getFileSystemClass

```
public static Class<? extends FileSystem> getFileSystemClass(String scheme,
                                                             Configuration conf)
    throws IOException
```

Throws:

IOException

getStatistics

`@Deprecated`
`public static Map<String,org.apache.hadoop.fs.FileSystem.Statistics> getStatistics()`
Deprecated. use `getAllStatistics()` instead
Get the Map of Statistics object indexed by URI Scheme.
Returns:
a Map having a key as URI scheme and value as Statistics object

getAllStatistics

`public static List<org.apache.hadoop.fs.FileSystem.Statistics> getAllStatistics()`
Return the FileSystem classes that have Statistics

getStatistics

`public static org.apache.hadoop.fs.FileSystem.Statistics getStatistics(String scheme, Class<? extends FileSystem> cls)`
Get the statistics for a particular file system
Parameters:
cls - the class to lookup
Returns:
a statistics object

clearStatistics

`public static void clearStatistics()`
Reset all statistics for all file systems

printStatistics

`public static void printStatistics() throws IOException`
Print all statistics for all file systems
Throws:
IOException

areSymlinksEnabled

`public static boolean areSymlinksEnabled()`

enableSymlinks

`public static void enableSymlinks()`